SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I B Ragava Prasath 192421026 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Q1. Online Banking System – Fund Transfer Failures at Boundary Values

1. Scenario analysis

The failures appear at the minimum and maximum transfer limits. This points to a **boundary-condition defect**: for example the code may check amount > min instead of amount >= min, or amount < max instead of amount <= max. Such an error is never exposed by arbitrary mid-range amounts. For this solution the following rules are assumed (illustrative values; the real ones come from the requirement document): minimum transfer Rs. 100, maximum transfer Rs. 50,000 per transaction, and the amount must not exceed the available balance.

2. Evaluation of the current test case design

- Test cases were not derived from the requirement ranges, so no equivalence classes were identified. The same valid class was tested repeatedly while the invalid classes (below minimum, above maximum, non-numeric, insufficient balance) were not tested.
- No boundary values were selected, although defects cluster at the edges of input ranges.
- Random testing gives no guarantee that the limits are ever chosen and has no defined expected result for each input.
- There is no traceability from requirement to test case, so the missing coverage was not visible.

3. Equivalence class partitioning (ECP)

Class	Input range	Type	Representative value	Expected result
EC1	Amount < 100	Invalid	50	Rejected: below minimum limit
EC2	100 ≤ amount ≤ 50,000	Valid	5,000	Transfer accepted
EC3	Amount > 50,000	Invalid	75,000	Rejected: exceeds maximum limit
EC4	Non-numeric, blank or negative	Invalid	abc, (blank), −500	Rejected: invalid format
EC5	Valid amount greater than balance	Invalid	Amount 20,000, balance 10,000	Rejected: insufficient funds

4. Boundary value analysis (BVA)

For every boundary the value just below, on, and just above the boundary is tested. This detects the wrong relational operator (> instead of >=) directly.

Test	Amount (Rs.)	Boundary checked	Expected result
B1	99	Just below minimum	Rejected
B2	100	Minimum	Accepted
B3	101	Just above minimum	Accepted
B4	49,999	Just below maximum	Accepted
B5	50,000	Maximum	Accepted
B6	50,001	Just above maximum	Rejected
B7	0 and −1	Invalid lower extreme	Rejected

Test	Amount (Rs.)	Boundary checked	Expected result
B8	Amount = balance (5,000 / 5,000)	Balance boundary	Accepted
B9	Amount = balance + 1 (5,001 / 5,000)	Balance boundary	Rejected: insufficient funds

5. Effectiveness of random testing

Random testing is cheap, easy to automate and can expose unexpected crashes or format problems. However, it is weak for this defect. If amounts are drawn uniformly from 1 to 10,00,000, the probability that one random test equals exactly 100 or 50,000 is only 2 in 1,000,000. Even 1,000 random tests would hit a boundary with a probability of about 0.2%. Random tests also have no oracle (the expected result must be worked out for every input), are not repeatable unless the seed is stored, and give no measure of coverage.

Decision: random testing should be **replaced** as the main technique and **retained only as a supplementary, seeded robustness test** that runs after the systematic tests.

6. Proposed structured and optimized approach

- **Step 1 – Requirement analysis:** list every limit and business rule (minimum, maximum, balance, daily limit, account status).
- **Step 2 – ECP:** one test per valid and invalid class (EC1 to EC5).
- **Step 3 – BVA:** three values at every boundary (B1 to B9).
- **Step 4 – Decision table:** combine amount validity, balance sufficiency and account status to cover rule interactions.
- **Step 5 – Error guessing:** duplicate click on Submit, network timeout during transfer, two simultaneous transfers from the same account.
- **Step 6 – Seeded random/fuzz tests and automated regression:** run all of the above after every change.

7. Justification

The 14 designed tests (5 ECP + 9 BVA) cover every class and every boundary, whereas thousands of random tests would still probably miss them. Each boundary defect type (> instead of >=, < instead of <=, off-by-one in the limit constant) is caught by at least one pair of adjacent tests such as B1/B2 or B5/B6. This gives maximum defect detection with the fewest tests and gives a traceable, repeatable test suite.

Q2. Healthcare Application – Post-Deployment Defects Due to Insufficient Coverage

1. Evaluation of the testing approach and gaps in test adequacy criteria

- The testing was driven only by normal (happy-path) scenarios; the safety-related branches such as the drug-allergy block and the paediatric dose limit were not exercised.
- No test adequacy criterion was defined. A criterion such as "100% branch coverage for safety-critical logic" would have exposed the gaps.
- No control flow graph (CFG) was built, so the missing paths were invisible to the team.
- Coverage was not measured, so the team could not tell how much of the module was actually executed.

2. Conceptual module and its control flow graph

A prescription-validation module is used as the conceptual example:

```
read age, weight, drugAllergy, dose    // node 1
if (drugAllergy)    // node 2
   block prescription;    // node 3
else {
   if (age < 12) maxDose = 10 * weight;    // nodes 4, 5
   else maxDose = 500;    // node 6
   if (dose > maxDose) flag overdose;    // nodes 7, 8
   else approve;    // node 9
}    // node 10 = exit
```

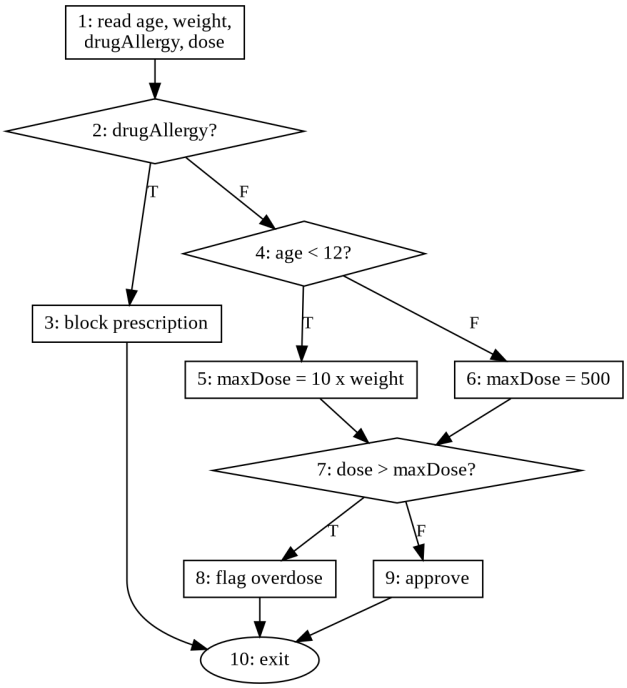


Fig. 1: Control flow graph of the prescription-validation module

Nodes N = 10, edges E = 12, predicate nodes = 3 (nodes 2, 4, 7). Cyclomatic complexity V(G) = E − N + 2 = 12 − 10 + 2 = **4**, which equals predicates + 1 = 3 + 1. The module has 5 feasible paths in total: 1 allergy path,

plus 2 × 2 combinations of the age and dose decisions.

3. Coverage achieved by the existing tests (assumed)

Assume the team ran only two happy-path tests: T-a (no allergy, age 30, dose 400, path 1-2-4-6-7-9-10) and T-b (no allergy, age 40, dose 600, path 1-2-4-6-7-8-10).

Coverage criterion	Calculation	Achieved
Statement coverage	8 of 10 nodes executed (nodes 3 and 5 never run)	80%
Branch coverage	8 of 12 edges executed (2→3, 3→10, 4→5, 5→7 never run)	66.7%
Path coverage	2 of 5 feasible paths executed	40%

Missing paths: the allergy-block path and the entire child-dose path. These carry the highest clinical risk, which explains the critical post-deployment defects.

4. Improved testing strategy

The basis path set (4 independent paths) is extended with the fifth path to reach complete path coverage, and the data values are chosen at the age and dose boundaries so that black-box boundary checks are combined with the structural coverage.

Test	Path	Input	Expected result
TP1	1-2-3-10	Allergy = Yes, age 30, dose 100	Prescription blocked
TP2	1-2-4-5-7-8-10	No allergy, age 8, weight 20, dose 250 (limit 200)	Overdose flagged
TP3	1-2-4-5-7-9-10	No allergy, age 11, weight 20, dose 200 (= limit)	Approved (boundary)
TP4	1-2-4-6-7-8-10	No allergy, age 30, dose 501 (limit 500)	Overdose flagged
TP5	1-2-4-6-7-9-10	No allergy, age 12, dose 500 (= limit)	Approved (age boundary 12)

With TP1 to TP5 the coverage becomes statement 100%, branch 100% (12 of 12 edges) and path 100% (5 of 5 paths). The adequacy criterion adopted is: 100% statement, 100% branch and all independent paths for every safety-related module, checked with a coverage tool before release.

5. Justification and effect on reliability and risk

- Every decision outcome is forced to both true and false, so no safety branch is left unexecuted.
- Boundary data (age 11/12, dose equal to the limit) catches wrong relational operators inside the covered branches.
- A measurable criterion makes the test gap visible before deployment, so risk is reduced early and cheaply.
- Reliability improves because the failures found post-deployment come from the untested paths, which are now executed in every regression run.

Q3. Retail E-Commerce System – Errors in Input Combinations (Price, Quantity, Discount)

1. Scenario analysis and limitations of the current black-box approach

Assumed rules (illustrative): price Rs. 1 to Rs. 10,000, quantity 1 to 100, discount 0% to 50%; total = price × quantity × (1 − discount). Testers appear to have validated each field separately with one fixed, valid value for the other fields. The limitations are:

- Single-field testing cannot find faults that appear only when two or three fields interact, for example an invalid discount together with a valid price, or the maximum price with the maximum quantity.
- Boundary values of each field were not combined with boundary values of the others.
- Invalid combinations were not tested systematically, so a message for only the first invalid field may be shown and the others ignored.
- Exhaustive testing is impossible: 10,000 prices × 100 quantities × 51 discounts = 51,000,000 valid combinations.

2. Equivalence class partitioning and boundary value analysis

Field	Valid class	Invalid classes	Boundary values (BVA)
Price	1 to 10,000	< 1, > 10,000, non-numeric	0, 1, 2, 9,999, 10,000, 10,001
Quantity	1 to 100	< 1, > 100, decimal, non-numeric	0, 1, 2, 99, 100, 101
Discount (%)	0 to 50	< 0, > 50, non-numeric	−1, 0, 1, 49, 50, 51

Robust BVA (one variable at an extreme value while the other two stay at nominal values: price 500, quantity 10, discount 10%) needs 6n + 1 = 6 × 3 + 1 = **19 tests**, instead of the 7^3 = 343 tests of worst-case robust BVA.

3. Decision table testing for combinations of field validity

Rule	Price valid	Quantity valid	Discount valid	Expected action
R1	Y	Y	Y	Calculate and display total
R2	Y	Y	N	Reject: invalid discount
R3	Y	N	Y	Reject: invalid quantity
R4	Y	N	N	Reject: invalid quantity and discount
R5	N	Y	Y	Reject: invalid price
R6	N	Y	N	Reject: invalid price and discount
R7	N	N	Y	Reject: invalid price and quantity
R8	N	N	N	Reject: all three fields reported

4. Revised test design strategy (selected interaction tests)

Test	Price	Quantity	Discount	Expected result
IT1	10,000	100	0%	Total 10,00,000.00 (maximum total; no overflow or display truncation)
IT2	10,000	100	50%	Total 5,00,000.00
IT3	1	1	50%	Total 0.50 (rounding to two decimals)
IT4	1	100	50%	Total 50.00
IT5	0	101	51%	Reject; all three errors are reported together

Revised strategy: (1) ECP per field, (2) 19 robust BVA tests, (3) decision-table rules R1 to R8 (R1 is covered by the BVA baseline, so 7 extra tests), (4) 4 to 5 interaction tests on maximum and minimum totals, and (5) pairwise combination tests if more fields (for example tax or coupon) are added. This is about 30 tests in total.

5. Justification

- Defect detection improves because every field boundary and every valid/invalid combination of the three fields is exercised at least once.
- Test completeness is demonstrated by a traceable table: every ECP class, every boundary and every decision rule maps to at least one test.
- About 30 designed tests replace an impossible 51 million exhaustive tests while still targeting the places where combination faults occur.

Q4. Software Module with Nested Conditions and Loops – Partial Path Coverage

1. Conceptual module

A module that adds positive array values (capped at a limit) and classifies the total is used to illustrate the analysis:

```
i = 1; total = 0;    // node 1
while (i <= n) {    // node 2
   if (a[i] > 0) {    // node 3
      if (a[i] > limit) total += limit;    // nodes 4, 5
      else total += a[i];    // node 6
   } else skipped++;    // node 7
   i++;    // node 8
}
if (total > 100) print "High";    // nodes 9, 10
else print "Normal";    // node 11
```

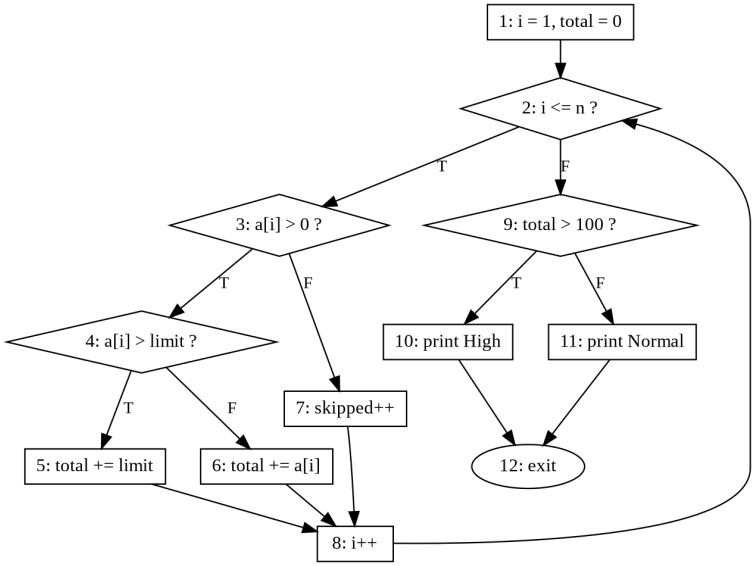


Fig. 2: Control flow graph of the nested condition and loop module

2. Gaps in the current white-box approach

- The CFG was not drawn completely: the loop-back edge 8 → 2 and the zero-iteration edge 2 → 9 were not treated as separate paths.
- Only paths in which every array element is positive and below the limit were exercised, so the nodes 5 and 7 (cap and skip) were rarely or never run.
- Because a loop creates an unbounded number of paths, the team could not state a clear coverage target and reported only "partial path coverage".

3. Independent paths and their impact on defect detection

Nodes N = 12, edges E = 15, predicate nodes = 4 (nodes 2, 3, 4, 9). V(G) = E − N + 2 = 15 − 12 + 2 = **5**, which equals predicates + 1. Five independent paths are therefore needed as a minimum:

Path	Sequence of nodes	Meaning
BP1	1-2-3-4-6-8-2-9-11-12	One positive value below the limit, result Normal (baseline)
BP2	1-2-9-11-12	Loop skipped (n = 0)
BP3	1-2-3-7-8-2-9-11-12	Non-positive value skipped
BP4	1-2-3-4-5-8-2-9-11-12	Value above the limit is capped
BP5	1-2-3-4-6-8-2-9-10-12	Total exceeds 100, result High

If any of these independent paths is not identified, the statements and decisions that belong only to it (capping logic, skip counter, zero-iteration handling, High classification) remain unverified; the defects hidden there appear only in production.

4. Complete path testing strategy

Because the loop makes the number of paths unbounded, complete coverage is defined as: all 5 basis paths + all edges (100% branch) + the standard loop tests (skip the loop, one pass, two passes, typical number of passes, n − 1, n and n + 1 passes).

Test	Input	Path	Expected outcome
T1	n = 0	BP2	total = 0, output Normal
T2	n = 1, a = [50], limit = 100	BP1	total = 50, output Normal
T3	n = 1, a = [−5]	BP3	skipped = 1, total = 0, output Normal
T4	n = 1, a = [150], limit = 100	BP4	total = 100 (capped), output Normal (100 is not > 100)
T5	n = 3, a = [60, 70, 80], limit = 100	BP5	total = 210, output High
T6	n = 2, a = [40, 70]	Loop twice	total = 110, output High (loop boundary, two passes)

5. Justification

The coverage criteria used are 100% statement, 100% branch and 100% basis-path coverage plus loop boundary testing. T1 to T5 give 15 of 15 edges, so every decision takes both outcomes. The expected outcomes are derived from the requirement (cap value, skip rule, threshold 100), not from the code, so defects in the logic are detected and not just executed.

Q5. Financial Application – Defects Persist Despite Black-Box and White-Box Testing

1. Critical analysis of the integration of test design strategies

- Black-box and white-box techniques were run as separate, unconnected activities; the structural coverage results were never used to improve the black-box test set.
- ECP and BVA were applied to the main input fields only. Hidden rules that exist in the code (for example an extra check for large amounts) were not in the specification, so they had no test.
- White-box testing was limited to statement coverage, which does not force both outcomes of decisions.
- There was no traceability between requirements, test cases and code coverage, and no exit criterion for test completion.

2. Evaluation of test adequacy criteria and coverage metrics

Criterion	Should have been applied	Likely gap
Functional (black box)	Every ECP class and every boundary value covered	Only valid classes and mid-range values tested
Statement coverage	Reported, but only as a minimum	Treated as the final adequacy measure
Branch / decision coverage	100% for financial calculations	Not measured
Path coverage (CFG)	All independent paths (cyclomatic complexity)	CFG not constructed for critical modules
Requirements coverage	Traceability matrix	Missing, so untested rules were not noticed

3. Worked illustration: loan eligibility module

Rule in the code: if score ≥ 700 and income ≥ 30,000 the loan is approved, but if the amount is above 5,00,000 it is sent to a manager; otherwise it is referred or rejected.

```
if (score >= 700) {
   if (income >= 30000) {
      if (amount > 500000) refer to manager;
      else approve;
   } else refer;
} else reject;
```

The code has 3 decisions, so V(G) = 3 + 1 = **4** paths: reject, refer (low income), manager review (large amount) and approve. A black-box suite designed with ECP/BVA on score and income only (699, 700, 701 and 29,999, 30,000, 30,001) executes three of the four paths; the manager-review path is never reached because the amount was not in the specification. This is exactly the type of defect that survives into production.

Test	Score	Income	Amount	Technique	Expected result
H1	699	30,000	1,00,000	BVA (score)	Reject
H2	700	29,999	1,00,000	BVA (score, income)	Refer
H3	700	30,000	1,00,000	BVA (both)	Approve
H4	701	30,001	1,00,000	ECP valid class	Approve
H5	700	30,000	5,00,001	Path testing (added from CFG)	Manager review

Test	Score	Income	Amount	Technique	Expected result
H6	700	30,000	5,00,000	Boundary on the path found by CFG	Approve

4. Improved hybrid testing strategy

Phase	Activity	Technique	Output / exit criterion
1	Derive functional tests from requirements	ECP, BVA, decision tables	Every class and boundary has a test
2	Build CFG of critical modules and compute V(G)	CFG, cyclomatic complexity	Number of independent paths known
3	Run phase 1 tests and measure coverage	Statement, branch, path coverage	Uncovered branches and paths listed
4	Add white-box tests for the uncovered paths	Basis path testing, MC/DC for compound conditions	100% branch and all basis paths covered
5	Add experience-based and regression tests	Error guessing, automated regression	Defect history covered, suite re-run on every change
6	Review and approve	Traceability matrix	Release only when the adequacy criteria are met

5. Justification and conclusion

Black-box tests make sure the specified behaviour is correct, and white-box tests make sure no implemented behaviour is left unexecuted. Using the coverage measured in phase 3 as feedback closes the gap between the two, which is why H5 and H6 could be found. Defect prevention is improved because the criteria (100% branch, all basis paths, all boundaries) are measurable and are checked before release. **Conclusion:** a hybrid strategy driven by explicit adequacy criteria and coverage feedback gives higher coverage, earlier detection and fewer production defects than either technique used alone.

Q6. HR Payroll System – Incorrect Bonus at Exactly 5, 10 and 15 Years

1. Analysis of the failure

Assumed bonus rules (illustrative): bonus slab by years of service: 0–4 years = 5%, 5–9 years = 10%, 10–14 years = 15%, 15 or more years = 20% of monthly salary. Performance rating 1–5: rating 1–2 = no bonus, rating 3 = slab percentage, rating 4–5 = slab percentage + 5%. Monthly salary used in the examples is Rs. 50,000.

Employees at **exactly** 5, 10 and 15 years are the lower edges of the slabs. They are paid wrongly because the boundary comparison is most likely coded with the wrong operator (years > 5 instead of years >= 5), so they fall into the lower slab (for example 5 years, rating 3: expected Rs. 5,000, paid Rs. 2,500). The fault is invisible when the test values are 2, 7, 12 and 20 years.

2. Evaluation of the current test design

- Test data were taken from the middle of each slab, never from the edges.
- No equivalence classes or boundary values were derived from the slab table.
- Testing appears to have relied on unscripted (exploratory) checks, which are not repeatable and do not guarantee that the slab limits were tried.

3. Equivalence class partitioning

Input	Valid classes	Invalid classes
Years of service	0–4, 5–9, 10–14, 15 and above	Negative values, non-integer / non-numeric
Performance rating	1–2 (poor), 3 (average), 4–5 (good)	< 1, > 5, non-numeric

4. Boundary value analysis – years of service (rating 3, salary Rs. 50,000)

Test	Years	Expected slab	Expected bonus (Rs.)
Y1	−1	Invalid	Rejected
Y2	0, 1, 4	5%	2,500
Y3	5, 6	10%	5,000
Y4	9	10%	5,000
Y5	10, 11	15%	7,500
Y6	14	15%	7,500
Y7	15, 16	20%	10,000

The values 4/5, 9/10 and 14/15 are the adjacent pairs that expose the incorrect operator at exactly 5, 10 and 15 years.

5. Boundary value analysis – rating (years = 5, slab 10%)

Test	Rating	Expected result
R1	0 and 6	Rejected (invalid rating)
R2	1 and 2	No bonus (Rs. 0)
R3	3	Rs. 5,000
R4	4 and 5	15% = Rs. 7,500

6. Combined boundary tests

Test	Years	Rating	Expected bonus
C1	5	4	15% = Rs. 7,500
C2	10	4	20% = Rs. 10,000
C3	15	5	25% = Rs. 12,500
C4	4	5	10% = Rs. 5,000
C5	15	2	Rs. 0

7. Why ECP and BVA are superior to exploratory testing for this financial logic

- **Boundaries are guaranteed to be tested:** the values 4, 5, 9, 10, 14 and 15 are chosen by rule; an explorer may or may not pick them.
- **Repeatable and auditable:** payroll is a financial and legal calculation, so the exact inputs and expected amounts must be documented and re-run after every change; exploratory sessions leave little evidence.
- **Precise expected results:** the correct amount can be calculated from the slab table in advance (oracle), so the wrong payout is detected immediately.
- **Efficient:** about 20 designed tests cover all slabs, all ratings and all edges, while exploratory testing has no coverage measure.
- Exploratory testing is still useful as a supplement for unexpected situations (for example mid-year joiners), but not as the primary technique for deterministic financial rules.

Q7. Laboratory Temperature Control System – Alarm Failure and MC/DC

1. Conceptual logic module

The control module reads the temperature and the status of two sensors, switches the heater or cooler, and raises an alarm if both sensors have failed or the temperature is above 40 °C:

```
if (temp < 18) heater ON;    // node 2, 3
else if (temp > 28) cooler ON;    // node 4, 5
else heater and cooler OFF;    // node 6
if ((s1Fault && s2Fault) || temp > 40) alarm ON;    // node 7, 8
else alarm OFF;    // node 9
```

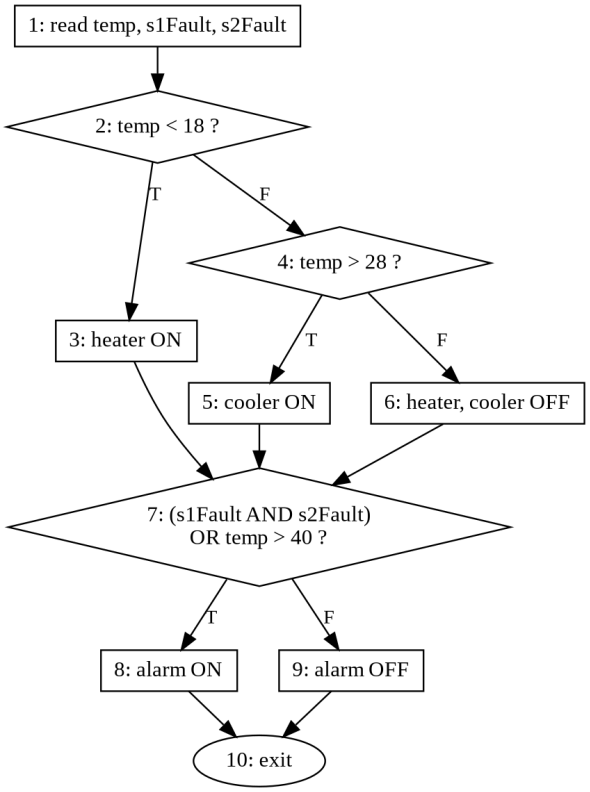


Fig. 3: Control flow graph of the temperature control module

N = 10, E = 12, decision nodes = 3, so V(G) = 12 − 10 + 2 = **4** at decision level (the compound decision in node 7 contains three conditions, so at condition level the complexity is 6).

2. Evaluation of the gaps in branch coverage

Assume the post-release tests covered only normal operating situations: temperature 15 (heater), 35 (cooler) and 22 (idle), all with healthy sensors and temperature below 40. The alarm decision was always false.

Criterion	Calculation	Result
Statement coverage	9 of 10 nodes (node 8, alarm ON, never executed)	90%
Branch coverage	10 of 12 edges (7→8 and 8→10 never executed)	83.3%
Condition outcomes	Conditions s1Fault, s2Fault and temp > 40 were never true	0 of 3 true outcomes

The high percentages hide the fact that the entire alarm path was never tested, which explains why the alarm did not trigger after release in the sensor-malfunction sequence.

3. White-box strategy for 100% MC/DC

Let A = s1Fault, B = s2Fault and C = (temp > 40). The decision is D = (A AND B) OR C. MC/DC requires every condition to be shown to change the decision outcome independently. For n = 3 conditions a minimum of n + 1 = 4 tests is enough.

Test	A (s1Fault)	B (s2Fault)	C (temp > 40)	D (alarm)	Independence shown for
M1	0	1	0	0	A with M3; C with M2
M2	0	1	1	1	C with M1
M3	1	1	0	1	A with M1; B with M4
M4	1	0	0	0	B with M3

The pairs are: A changes (M1, M3) with B and C fixed; B changes (M3, M4) with A and C fixed; C changes (M1, M2) with A and B fixed. To also cover the heater and cooler branches, the same four tests are given suitable temperatures:

Test	Temperature	s1Fault / s2Fault	Expected result
M1	15 °C	No / Yes	Heater ON, alarm OFF
M2	45 °C	No / Yes	Cooler ON, alarm ON
M3	35 °C	Yes / Yes	Cooler ON, alarm ON (both sensors failed)
M4	22 °C	Yes / No	Idle, alarm OFF

These four tests also give 100% statement and 100% branch coverage (12 of 12 edges): heater (M1), cooler (M2, M3), idle (M4), alarm ON (M2, M3), alarm OFF (M1, M4).

4. Impact on safety-critical reliability

- MC/DC shows that **each** sensor condition can independently trigger the alarm, so a masked or missing condition (for example the code using OR where AND was intended) is detected, which branch coverage alone cannot guarantee.
- It is the coverage level required for the highest criticality software in aviation (DO-178C level A) and is widely used for other safety-critical systems.
- The suite is small (n + 1 tests per decision) instead of 2^n combinations, so it is practical to execute and to maintain.
- The alarm path is verified with defined expected results, which directly reduces the risk of an undetected unsafe laboratory condition.

Q8. Travel Booking Portal – Negative Total Amount with Multiple Promo Codes

1. Limitations of testing the inputs in isolation

Assumed rules (illustrative): promo A gives 20% of the base price, promo B gives a flat Rs. 500, and the premium user tier gives an extra 10%. Discounts are calculated on the original price and added together. The total payable must never be below the minimum booking fee of Rs. 100.

- Each promo code, price and tier works correctly when tested alone, but the defect appears only when several discounts are combined.
- The result depends on the price: a flat discount larger than the discounted price produces a negative total only for low prices.
- Testing each input in isolation cannot reveal this interaction; the number of valid and invalid combinations must be modelled explicitly.

2. Decision table (conditions and actions)

Conditions: C1 = promo A applied, C2 = promo B applied, C3 = premium tier, C4 = discounts ≥ price (total would be zero or negative). Action: calculate the total, and apply the minimum fee if C4 is true.

Rule	C1	C2	C3	C4	Expected action
R1	N	N	N	N	No discount
R2	N	N	Y	N	10% discount
R3	Y	N	N	N	20% discount
R4	Y	N	Y	N	30% discount
R5	N	Y	N	N	Flat Rs. 500
R6	N	Y	Y	N	Rs. 500 + 10%
R7	Y	Y	N	N	20% + Rs. 500
R8	Y	Y	Y	N	30% + Rs. 500
R9	N	Y	N	Y	Discount capped, total = minimum fee
R10	N	Y	Y	Y	Discount capped, total = minimum fee
R11	Y	Y	N	Y	Discount capped, total = minimum fee
R12	Y	Y	Y	Y	Discount capped, total = minimum fee

C4 can be true only when the flat promo B is applied (C2 = Y) because the percentage discounts alone never exceed 30%. Therefore the 16 possible combinations reduce to 12 meaningful rules.

3. Optimized test set

Test	Rule	Price (Rs.)	Promos / tier	Expected total (Rs.)
T1	R1	5,000	None / regular	5,000
T2	R2	5,000	None / premium	4,500
T3	R3	5,000	A / regular	4,000
T4	R4	5,000	A / premium	3,500
T5	R5	5,000	B / regular	4,500
T6	R6	5,000	B / premium	4,000
T7	R7	5,000	A + B / regular	3,500
T8	R8	5,000	A + B / premium	3,000
T9	R9	400	B / regular	100 (400 − 500 = −100, capped)
T10	R10	400	B / premium	100 (capped)
T11	R11	400	A + B / regular	100 (capped)
T12	R12	400	A + B / premium	100 (capped)
T13	Boundary	500	B / regular	100 (500 − 500 = 0, capped)
T14	Boundary	600	B / regular	100 (exactly the minimum, not capped)

4. Justification

- The 14 tests cover every rule of the decision table, including all combinations that can make the total negative, and the price boundary where the flat discount equals the price (T13, T14).
- Redundant tests are removed: the full set of price classes × promo combinations × tiers would be 24 or more cases, and rules with the same action are merged.
- Defect detection is maximized because the defect-prone region (low price with the flat promo) is tested with every tier and promo combination, while minimum redundancy keeps the suite small and maintainable.

Q9. Legacy Database Migration Tool – High Statement Coverage but Infinite Loops

1. Conceptual module

```
i = 0;    // node 1
while (i < records) {    // node 2
   j = 0;    // node 3
   while (j < fields) {    // node 4
      if (isValid(rec[i][j])) j = j + 1;    // nodes 5, 6
      else if (repair(rec[i][j]) == OK)    // node 7
                        j = j + 1;    // node 8
      // no else: when repair fails, j is unchanged
   }
   i = i + 1;    // node 9
}
```

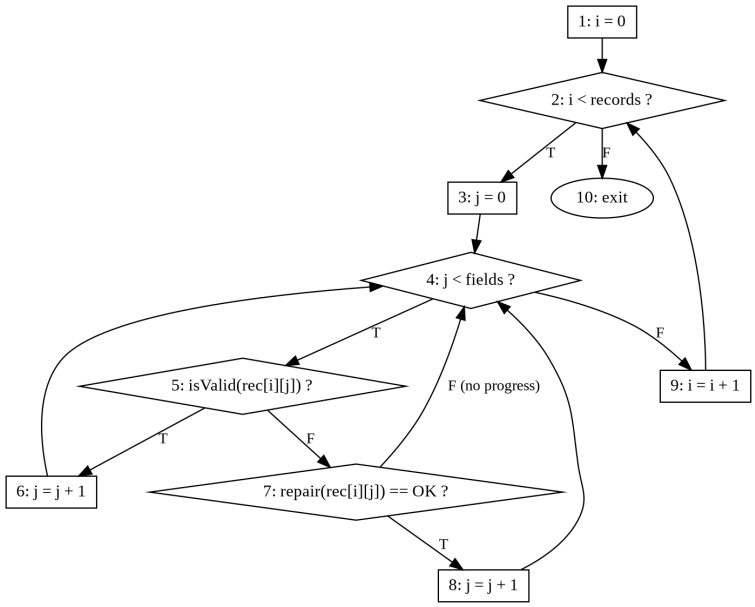


Fig. 4: Control flow graph of the nested loop structure

2. Why statement coverage was insufficient

The test data contained valid fields and invalid fields that could be repaired, so every statement (nodes 1 to 9) was executed and statement coverage reached 100%. The faulty case is the **false outcome of node 7** (repair fails): this branch has no statement of its own (there is no else part), so executing it adds nothing to statement coverage. In that case j is not changed, the inner loop condition stays true and the loop never ends. Statement coverage measures executed statements, not decision outcomes or loop behaviour, so it could not reveal this defect.

3. Cyclomatic complexity

N = 10 nodes, E = 13 edges, predicate nodes = 4 (nodes 2, 4, 5, 7).

V(G) = E − N + 2 = 13 − 10 + 2 = **5**, and V(G) = predicates + 1 = 4 + 1 = **5**. At least 5 independent paths must be tested. A value of 5 is within the commonly accepted limit of 10, but the nested loops and the missing else

make the module risky, so it should also be refactored with a retry limit.

4. Path testing strategy – independent paths

Path	Sequence of nodes	Scenario
BP1	1-2-3-4-5-6-4-9-2-10	One record with one valid field (baseline)
BP2	1-2-10	No records (outer loop skipped)
BP3	1-2-3-4-9-2-10	Record with no fields (inner loop skipped)
BP4	1-2-3-4-5-7-8-4-9-2-10	Invalid field repaired successfully
BP5	1-2-3-4-5-7-4 … (repeats)	Invalid field, repair fails: returns to node 4 without progress

BP5 is the path that statement coverage missed. It must be executed with a timeout; the expected outcome is that the record is rejected and logged after a maximum number of retries, whereas the present code hangs.

5. Loop boundary conditions

Test	Records	Fields per record	Repair result	Expected outcome
L1	0	–	–	Exits immediately, nothing migrated
L2	1	0	–	Record accepted, inner loop skipped
L3	1	1	Valid field	Migrated
L4	2	2	Valid and repaired	Both records migrated
L5	1	1	Repair fails	Rejected and logged after retry limit; no infinite loop (5 s timeout)
L6	Maximum (e.g. 10^6)	Maximum	Valid	Completes within time; no overflow of i or j
L7	Maximum + 1 / outer = max, inner = min	0 or 1	Valid	Nested loop rule: test inner loop at minimum while outer loop is at maximum

6. Justification using cyclomatic complexity

- V(G) = 5 gives the minimum number of tests that cover all independent paths; BP1 to BP5 cover all 13 edges, so branch coverage becomes 100% and not only statement coverage.
- Loop boundary tests (zero, one, two, maximum passes) target the usual loop defects.
- High complexity indicates risk; the metric also tells the team where to concentrate the tests and when to refactor.

Q10. Mobile Food Delivery App – Real-Time Tracker Stuck After Driver Cancellation

1. Evaluation of the current black-box strategy

ECP and BVA choose input values and therefore treat every request independently. The tracker, however, is a state-based workflow in which the result of an event depends on the current state and on the history of earlier events. The tests covered the normal sequence Ordered → Preparing → Out for Delivery → Delivered and customer cancellation, but not the event "driver cancels while Out for Delivery". Since no state existed for this situation, the app had no valid next state and remained stuck.

2. State transition model (redesigned)

A "Reassigning Driver" state is added, in addition to Cancelled. Assumed rule: the customer cannot cancel once the food is out for delivery.

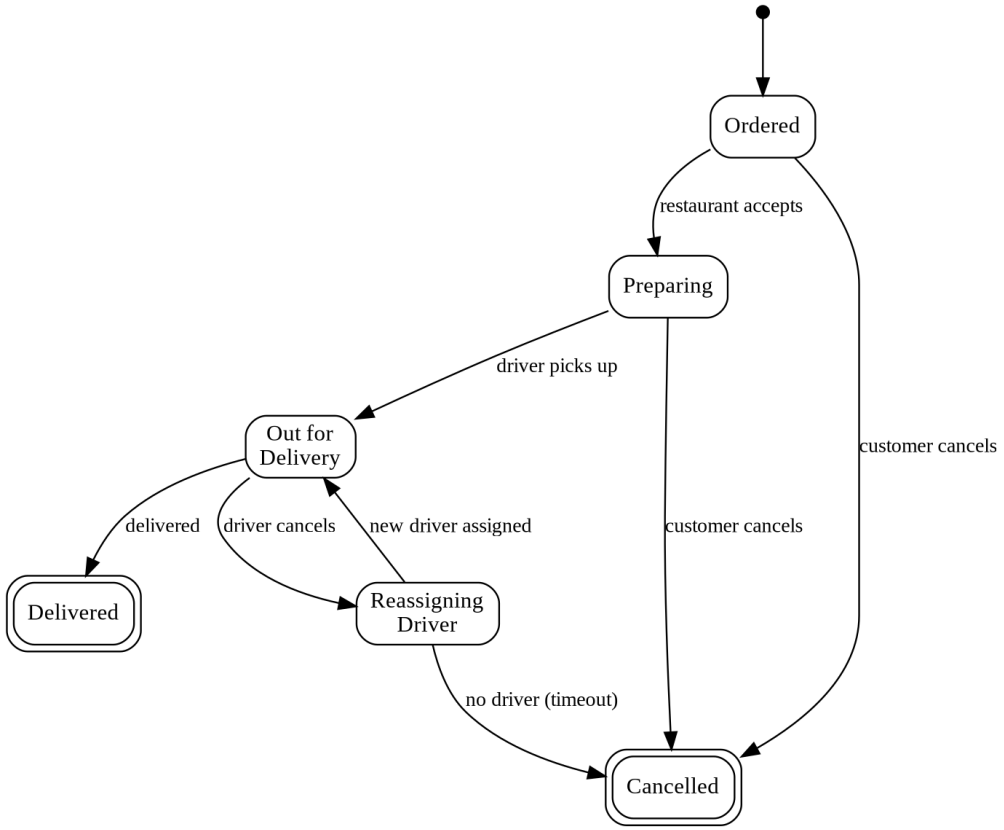


Fig. 5: State transition diagram of the Real-time Tracker

3. State transition table

Current state	Restaurant accepts	Driver picks up	Delivered	Customer cancels	Driver cancels	New driver / timeout
Ordered	Preparing	Invalid	Invalid	Cancelled	Invalid	Invalid
Preparing	Invalid	Out for Delivery	Invalid	Cancelled	Invalid	Invalid
Out for Delivery	Invalid	Invalid	Delivered	Invalid (rejected)	Reassigning	Invalid
Reassigning	Invalid	Invalid	Invalid	Cancelled	Invalid	Out for Delivery / Cancelled
Delivered	Invalid	Invalid	Invalid	Invalid	Invalid	Invalid
Cancelled	Invalid	Invalid	Invalid	Invalid	Invalid	Invalid

4. Redesigned test scenarios

Test	Sequence of events	Expected final state / behaviour
TC1	Accept, pick up, delivered	Delivered (full normal path)
TC2	Customer cancels in Ordered	Cancelled
TC3	Accept, customer cancels in Preparing	Cancelled, refund issued
TC4	Accept, pick up, driver cancels	Reassigning Driver (not stuck); customer notified
TC5	TC4, then new driver assigned, delivered	Out for Delivery, then Delivered
TC6	TC4, then no driver within timeout	Cancelled, refund issued
TC7	Driver cancels twice in succession (Out → Reassigning → Out → Reassigning)	Reassigning each time; no state corruption
TC8	Customer cancels while Out for Delivery	Rejected with message (assumed rule); state unchanged
TC9	Cancel after Delivered	Rejected; state remains Delivered
TC10	Driver picks up in Ordered (skipping Preparing)	Rejected; state remains Ordered
TC11	Restaurant accepts in Cancelled	Rejected; state remains Cancelled

Coverage achieved: all 6 states, all 8 valid transitions and the invalid transitions of the table (0-switch coverage), plus the 1-switch sequences TC5 to TC7.

5. Justification – gaps that ECP and BVA miss

- The same input ("driver cancels") is valid in only one state; its effect depends on the state, which ECP and BVA cannot express.
- Defects in dynamic workflows occur in sequences of events (cancel then reassign then cancel again), but ECP and BVA design single-input tests.
- State transition testing makes missing states and missing transitions visible when the model is drawn, so the "stuck" behaviour is found at design time.
- Invalid transitions are tested explicitly, so illegal state changes are prevented as well.

Q11. Cybersecurity Authentication Module – Brute-Force Vulnerabilities

1. Critical assessment of the black-box and white-box methods used

- Black-box tests checked the normal outcomes only (correct and incorrect password, valid OTP, trusted and untrusted IP) one factor at a time. Attack behaviour (many attempts, parallel requests, reuse) was not tested.
- White-box testing, if any, measured statement coverage; it does not show that every decision path has a protective counter or limit.
- The two approaches were not connected: the code paths that process failed attempts were never analysed together with realistic attack patterns.
- Security-specific requirements (lock-out, rate limiting, OTP expiry) had no explicit test cases or coverage criteria.

2. Formal path analysis of the authentication flow

Conceptual flow: (1) receive user name and password; (2) if the password is wrong, increase the failure counter and lock the account when the counter reaches 5; (3) if the password is correct, check the OTP (valid and within 30 seconds); (4) if the OTP is correct, check whether the IP location is trusted; (5) grant access, or ask for step-up verification if the IP is not trusted. There are 4 decisions (password, lock limit, OTP, IP), so V(G) = 4 + 1 = **5** independent paths:

Path	Scenario	Security question raised by the path analysis
P1	Wrong password, counter below limit	Is the counter increased and is the error message generic?
P2	Wrong password, counter reaches the limit	Is the account locked and for how long?
P3	Correct password, wrong or expired OTP	**Is the counter increased here?** In the assumed code it is not, so a 6-digit OTP can be guessed (10^6 combinations) without lock-out
P4	Correct password and OTP, untrusted IP	Is step-up verification forced and not bypassed?
P5	All factors correct, trusted IP	Access granted, counter reset, event logged

3. Error guessing – attack-oriented test ideas

Error guessing uses the tester's experience of how authentication modules usually fail. The list below is turned into test cases and attached to the path where each could occur:

Test	Error-guess scenario	Related path	Expected secure behaviour
E1	Wrong password 4, 5 and 6 times in a row	P1, P2	Lock at the 5th failure; the 6th attempt is refused even with the right password
E2	Correct password, many wrong OTPs (e.g. 100 attempts)	P3	OTP attempts are counted; the session is locked after the limit
E3	OTP used at 29 s, 30 s and 31 s (timing boundary)	P3	Accepted at 29 s and 30 s (as per requirement), rejected at 31 s

Test	Error-guess scenario	Related path	Expected secure behaviour
E4	Re-using an OTP that was already accepted	P3	Rejected (one-time use)
E5	Failures spread over different IPs or with a changed header	P1, P2	Counter is per account, not per IP; header values are not trusted
E6	Many parallel login requests (race condition)	P2	Counter is updated atomically; no extra attempts get through
E7	Unknown user vs wrong password	P1	Same message and similar response time (no user enumeration)
E8	Common-password list, empty and very long passwords	P1	Weak passwords rejected; no crash or truncation

4. Proposed hybrid testing strategy

- **Step 1:** build the CFG, list the 5 independent paths and write one functional test per path (formal path analysis).
- **Step 2:** for each path, apply error guessing to list the attack patterns that could reach it (E1 to E8) and add the tests.
- **Step 3:** measure branch coverage and verify that every failure branch contains a counter, limit or lock (security adequacy criterion).
- **Step 4:** run automated attack simulations (scripted repeated attempts and parallel requests) in a test environment and monitor the logs and alerts.
- **Step 5:** add every defect found to the regression suite.

5. Justification – effect on the security posture

Path analysis gives systematic and measurable coverage of the code (it found the unprotected OTP path P3), while error guessing adds the attacker's point of view that a specification-based technique does not contain. Together they ensure that every authentication path is limited by a counter, that time-based and one-time rules are verified at their boundaries, and that concurrency and spoofing cases are checked. This closes the brute-force weaknesses, and the regression suite keeps them closed after future changes.

Q12. E-Learning Platform – Tiered Access and Upgrade on the Day of Expiry

1. Evaluation of the adequacy of the current testing criteria

- Tests covered active subscriptions and clearly expired subscriptions, but the **expiry day itself** (the boundary between active and expired) was not tested.
- Plan changes (upgrades) were tested only in the middle of a subscription period, not at the end.
- No coverage criterion was defined for the combination of plan, date and course type, so the missing combination was not noticed.
- Time-related details (end of day, time zone, 23:59:59 versus 00:00:00) were not specified or tested.

2. Equivalence class partitioning

Input	**Equivalence classes**
Date of upgrade relative to expiry	D1: well before expiry (more than 1 day left) D2: one day before expiry D3: **on the expiry day** (boundary) D4: one day after expiry D5: well after expiry
Current subscription	S1: Free, S2: Basic, S3: Premium (and invalid / unknown plan)
Upgrade target	U1: Free → Basic, U2: Free → Premium, U3: Basic → Premium
Course type	C1: free course, C2: Basic course, C3: Premium course

Expected access rules: Free = free courses; Basic = free and Basic courses; Premium = all courses. An upgrade requested on or before the expiry day must give the new access immediately, without a gap, and the new end date must be calculated correctly. After the grace period/expiry the user is treated as Free until the upgrade is paid (assumed rule).

3. Structured test design

Test	Current plan	Date of upgrade	Action	Expected result
T1	Basic	D1	Upgrade to Premium	Premium access at once; new end date set
T2	Basic	D2 (expiry − 1 day)	Upgrade to Premium	Premium access, no gap
T3	Basic	D3 (expiry day, 10:00)	Upgrade to Premium	Premium access, no loss of access, no double charge
T4	Basic	D3 (23:59:59)	Upgrade to Premium	Same as T3 (last second of the expiry day)
T5	Basic	D4 (00:00:00 next day)	Upgrade to Premium	Access was Free in the gap; Premium on payment, no error
T6	Basic	D5	Upgrade to Premium	Treated as new subscription
T7	Free	D3 (not applicable: no expiry)	Upgrade to Basic	Basic access immediately
T8	Premium	D3	Renew Premium	Continuous Premium access, end date extended
T9	Basic	D3	Open a Premium course just before upgrading	Denied before, allowed after the upgrade
T10	Basic	D3 (user in a different time zone)	Upgrade to Premium	Expiry evaluated in one defined time zone; same result as T3
T11	Unknown plan value	D3	Upgrade	Rejected with a clear error message

4. Justification of completeness

- Every date class (D1 to D5), every plan and every upgrade path is used at least once, and the boundary (expiry day) is tested at its start, middle and last second.
- The access matrix (plan × course type) is verified before and after the upgrade (T9), so both the transition and the resulting permissions are checked.
- The tests are derived from the equivalence classes and not from guesses, so the missing "upgrade on expiry day" case cannot be left out again.
- Seamless transitions are ensured by verifying that there is no gap in access, no double charge and a correct new end date for every upgrade path.

Q13. Smart-Home Lighting Controller – Risks of Statement Coverage Only

1. Conceptual module

```
switch (timeOfDay) {
   case DAY: level = 0; break;    // case 1
   case EVENING: level = 60; break;    // case 2
   case NIGHT: level = 30; break;    // case 3
   default: level = 100;    // case 4 (fail-safe)
}
if (occupied && (lux < 300 || override))
   setLevel(level);
else
   setLevel(0);
```

2. Risks of relying only on statement coverage

- Statement coverage is satisfied as soon as each line has been executed once; it does not require that every case of the switch, or both outcomes of the if, are taken.
- The Boolean expression is evaluated with short-circuit logic: if lux < 300 is true, override is never evaluated. A fault in the override condition can therefore remain hidden although all statements were executed.
- The default (fail-safe) case is executed only for unexpected sensor or clock values; a test set that does not force it passes statement coverage in the other cases while leaving the safety behaviour untested.
- For a controller, an untested logical path becomes a hardware-software integration failure (for example lights fully on at night, or off when the room is occupied) that only appears in the field.

3. Branch and condition coverage

Required branch outcomes: switch cases DAY, EVENING, NIGHT, default (4) and if true/false (2) = **6 branch outcomes**. Atomic conditions: occupied, lux < 300 and override, each true and false = **6 condition outcomes**.

Test	timeOfDay	occupied	lux	override	Decision	Expected result
T1	DAY	T	200	F	True (lux < 300)	setLevel(0)
T2	EVENING	T	500	T	True (override)	setLevel(60)
T3	NIGHT	T	500	F	False	setLevel(0)
T4	Invalid value	F	100	F	False (not occupied)	setLevel(0); no crash
T5	Invalid value	T	100	F	True	setLevel(100), fail-safe level

Coverage obtained: switch cases DAY (T1), EVENING (T2), NIGHT (T3), default (T4, T5); if true (T1, T2, T5) and false (T3, T4) = 6 of 6 branch outcomes. Conditions: occupied true (T1, T2, T3, T5) and false (T4); lux < 300 true (T1) and false (T2, T3); override true (T2) and false (T3) = 6 of 6 condition outcomes. The missing logical paths found by this analysis are the override branch (never evaluated with short-circuit logic), the default case and the not-occupied case.

4. Improved white-box strategy

- Use **branch coverage (100%)** and **condition coverage (100%)** as the minimum criteria; for the compound expression add MC/DC (4 tests for three conditions) if the controller is considered safety-relevant.
- Add boundary tests for the sensor threshold: lux = 299, 300 and 301.
- Add integration-oriented tests: invalid or missing sensor value, sensor reading outside the physical range, and a rapid change of lux values.
- Measure coverage automatically in every build and fail the build if the criteria are not met.

5. Justification

Branch and condition coverage force each decision and each condition to take both outcomes, so the hidden override logic, the fail-safe case and the unoccupied case are executed with known expected results. The hardware-software integration risk is reduced because the behaviour for every combination of sensor state and mode that the software can see is verified in advance; the remaining risk is limited to the physical hardware, which is tested separately.

Q14. Logistics Management System – Zero and Negative Weight Values

1. Analysis of the failure of the current input validation testing

Assumed rules (illustrative): valid weight 0.1 kg to 50 kg, zones 1 to 3 (Rs. 10, 20 and 30 per kg) and delivery speed Standard (×1) or Express (×1.5). Cost = weight × zone rate × speed factor.

- The tests contained only positive, realistic weights, so the validation code was never exercised with zero or negative values.
- Without validation, a negative weight produces a negative shipping cost (a credit or refund) and zero weight produces a free shipment; neither case was noticed.
- The tests were limited to valid inputs (positive testing); there was no negative testing and no robustness testing at the invalid side of the boundaries.

2. Boundary value analysis with robustness testing

Robustness testing extends BVA to the values just **outside** the valid range. For the weight boundaries 0.1 and 50 the tested values are: below minimum, minimum, just above minimum, nominal, just below maximum, maximum and above maximum.

Test	Weight (kg)	Zone / speed	Boundary	Expected result
W1	−5	2 / Standard	Invalid (negative)	Rejected: weight must be between 0.1 and 50 kg
W2	−0.1	2 / Standard	Invalid (negative)	Rejected, no cost calculated
W3	0	2 / Standard	Invalid (zero)	Rejected, not a free shipment
W4	0.09	2 / Standard	Just below minimum	Rejected
W5	0.1	1 / Standard	Minimum	Accepted, cost Rs. 1.00
W6	0.11	1 / Standard	Just above minimum	Accepted, cost Rs. 1.10
W7	25	2 / Standard	Nominal	Accepted, cost Rs. 500.00
W8	49.9	3 / Express	Just below maximum	Accepted, cost Rs. 2,245.50
W9	50	3 / Express	Maximum	Accepted, cost Rs. 2,250.00
W10	50.1	3 / Express	Just above maximum	Rejected
W11	abc, blank, 99999999	1 / Standard	Invalid type / extreme	Rejected with an error message; no crash

Zone and speed are covered by equivalence classes: every zone (1, 2, 3) and both speeds are used with the valid weights, and invalid zones (0, 4) and speeds are tested as separate invalid classes. Invalid combinations are expected to give the same validation error for weight regardless of zone and speed.

3. Why negative scenarios are necessary for robust system performance

- **Financial protection:** an unchecked negative weight gives a negative cost and a zero weight gives free service, so the company loses money or can be exploited.
- **Data integrity:** invalid values must not be saved in orders, invoices or route plans, where they would cause failures later in the process.

- **Robustness:** real users, other systems and attackers send wrong data; the software must reject it with a clear message and continue to work, not crash or calculate a wrong result.
- **Coverage of the input space:** the invalid inputs are far more numerous than the valid ones; testing only valid inputs checks only a small part of the behaviour.
- **Security:** missing validation of extreme or malformed values is a common starting point for injection and overflow problems.

4. Conclusion

Boundary value analysis with robustness testing detects the missing validation at the exact edges (0.09, 0.1, 50, 50.1) and the invalid values (zero and negative), and the negative test cases give evidence that the system fails safely. The test suite of 11 weight tests plus the zone and speed class tests is small, repeatable and traceable to the validation requirement.

Q15. Medical Imaging Software – Inconsistent Results When the Data Size Is Prime

1. Evaluation of the reliance on random testing

- **No oracle:** random data sizes do not come with an expected result, so an inconsistent result is hard to recognize.
- **Low and uncontrolled probability of the failing case:** primes near 10^6 are about 1 in 14 numbers ($1/\ln 10^6 \approx 7\%$), and the failure also depends on the number of parallel workers and on n being smaller than the worker count. The random test cannot guarantee that this combination is run.
- **No coverage measure:** random tests say nothing about which paths of the splitting logic were executed.
- **Non-repeatable:** parallel programs plus random data make failures difficult to reproduce unless the seed and thread count are recorded.
- The observation "fails only for prime n" points to the real cause: a prime n is never divisible by w ($1 < w < n$), so the **remainder handling** of the work-splitting code is used only for prime sizes. This is a structural defect that structural (white-box) testing finds directly.

2. Conceptual module and control flow graph

```
read n, w;    // n data elements, w worker threads
chunk = n / w;
for (t = 0; t < w; t++) {
   start = t * chunk;
   end = start + chunk;
   if (t == w - 1 && n % w != 0)    // last worker takes the remainder
      end = n;
   process(start, end);
}
combine results;
```

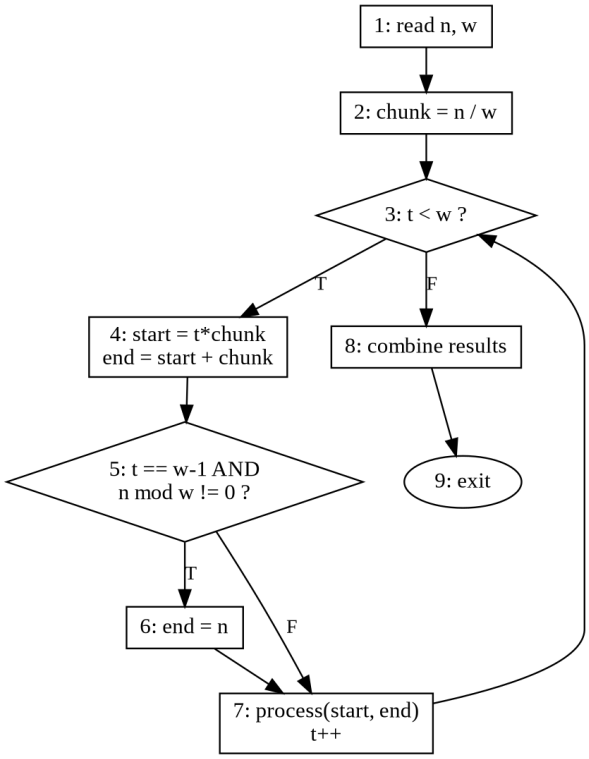


Fig. 6: Control flow graph of the work-splitting module

N = 9, E = 10, predicate nodes = 2 (nodes 3 and 5). V(G) = E − N + 2 = 10 − 9 + 2 = **3**, equal to predicates + 1. If the compound condition in node 5 is counted as two conditions, the complexity is 4.

3. Basis path testing

Path	Sequence of nodes	Scenario
BP1	1-2-3-4-5-7-3-8-9	Size divisible by the worker count (remainder not needed), baseline; the loop-back repeats the body w times
BP2	1-2-3-8-9	Loop executed zero times (w = 0): must be rejected before chunk = n / w
BP3	1-2-3-4-5-6-7-3-8-9	Remainder exists, the last worker takes it (typical prime size)

4. Structured white-box test set

Test	n	w	Path / purpose	Expected result
TC1	1000	0	BP2, zero iterations	Rejected with an error message (no division by zero)
TC2	8	4	BP1, divisible	Each worker has 2 elements; result equals the sequential result
TC3	7	4	BP3, prime n with remainder 3	Last worker handles elements 4 to 6; all 7 elements processed exactly once
TC4	2	4	n < w, prime (chunk = 0)	Only the last worker has elements 0 and 1; none are lost or duplicated
TC5	5	5	Prime n equal to w	Each worker processes exactly one element
TC6	9,973	8	Large prime, remainder 5	Result equals the sequential result
TC7	1	1 and 2	Smallest size	Correct result with one and with two workers
TC8	10,000 (not prime)	3	BP3 with a non-prime n (remainder 1)	Workers 0 and 1 end at start + chunk; worker 2 ends at n; the remainder defect is not limited to primes

Condition coverage of node 5: (t == w − 1) is true for the last worker and false for the others (TC3), and (n % w != 0) is true (TC3) and false (TC2). Branch coverage is 100% (10 of 10 edges); every decision outcome is exercised.

5. Justification – mathematical accuracy and system stability

- **Mathematical accuracy:** each parallel result is compared with a trusted sequential reference implementation (oracle). Floating-point sums are compared with a tolerance, since the order of addition differs. Invariants are checked: the sum of the chunk sizes equals n, and every index is processed exactly once with no overlap.
- Basis path testing guarantees that the remainder-handling path, which only prime (non-divisible) sizes reach, is executed with known expected results.
- **System stability:** repeat the tests many times and with different worker counts to detect race conditions; run with a thread sanitizer and a memory checker; test the large-size and zero-size limits.
- Together these tests give a repeatable, measurable verification of the splitting logic, which is more reliable than random testing of data sizes.